Adventist University of Central Africa

Course: Big Data Analytics

# Final Exam Project-Report

Date: 27th June 2026

MANIRAGUHA JEAN DE DIEU

ID: 101189

# Contents

# 1. SYSTEM ARCHITECTURE OVERVIEW

This project implements a polyglot persistence architecture for e-commerce analytics, combining three technologies chosen for complementary strengths rather than redundancy:

- MongoDB (document model): stores user profiles, product catalog, and transaction documents where data is naturally nested and read as a whole (e.g., a transaction with its line items, or a product with its price history).
- HBase (wide-column model): stores high-volume, time-series session and product-metric data where the access pattern is narrow scans by row-key prefix (e.g., "give me user X's recent sessions" or "give me product Y's daily metrics over a date range").
- Apache Spark: performs the heavy distributed computation that spans both stores — batch cleaning, market-basket analysis, cohort analysis, and the cross-system CLV integration query — because neither MongoDB nor HBase is designed to efficiently join across each other natively.

The diagram below summarizes the data flow: raw JSON (simulating an ingestion pipeline) is loaded into MongoDB and HBase according to the schema decisions in Section 2; Spark reads from both (here, via their underlying JSON exports, standing in for live connectors) to perform analytics that neither store could do alone.

| Component | Primary Role | Data Stored | Access Pattern |
|---|---|---|---|
| MongoDB 7.0 | Document store for rich, nested data | Users, Products, Categories, Transactions | Complex aggregation pipelines |
| HBase (dajobe) | Wide-column store for time-series data | User session streams, Product daily metrics | Fast prefix & range scans by row key |
| Apache Spark 3.5.6 | Distributed processing engine | Reads from JSON/Parquet; joins across sources | Batch ETL, SQL analytics, ML-style scoring |

## 1.1 Data Flow

- The dataset generator (dataset_generator.py) produces five JSON files: users, products, categories, transactions, and sessions.
- MongoDB ingests users, products, categories, transactions, and sessions — covering all rich-document and aggregation query needs.
- HBase ingests a processed subset of sessions (user_sessions table) and pre-aggregated product metrics (product_daily_metrics table).
- Apache Spark reads directly from JSON files (and Parquet for intermediate results), performing cleaning, transformation, SQL analytics, and cross-system CLV estimation.
- All three services run as Docker containers (ecommerce-mongo, ecommerce-hbase, ecommerce-spark) orchestrated by a single docker-compose.yml.

**E-commerce-project**

- Data : to generate Dataset and generate jsonfiles
- Mongodb : loader + 2 aggregation pipelines
- Hbase: schema, loader, query examples
- Spark: bath cleaning, affinity/cohort, spark sql
- Integration: clv cross-system integration query
- Visualization: cart generation script and output pngs
- Doker-compose.yml: spins up mongodb+hbase+spark
- Report: technical report

## 1.2 Dataset Scale

The dataset was deliberately sized for reproducibility on a single laptop:

- 500 users, 300 products, 25 categories, 3,000 transactions, 6,000 sessions (90-day window).
- This is a scaled-down version of the suggested volumes (10,000 users / 500,000 transactions). The schema, relationships, and generation logic are unchanged — the system design is identical and scales linearly to production volumes by adding Spark workers and HBase region servers.

# 2. Data Modeling Decisions and Rationale

## 2.1 MongoDB Schema Design

Three collections were implemented, each designed around how the data is actually read, not just how it's structured:

| Collection | Embedding Decision | Justification |
|---|---|---|
| products | price_history embedded as array | Always read together with the product; small, bounded size; avoids a join for a field that's always needed alongside the base product. |
| users | (total_spent, order_count, last_purchase_date) computed and embedded at load time | Avoids a transactions join for common dashboard-style reads like "show top spenders"; trades write-time computation for read-time speed, appropriate since user summaries change far less often than they're read. |
| transactions | line items embedded as array | Line items have no independent existence outside their parent transaction and are always written/read as a unit — a classic case for embedding over referencing |

Categories were also embedded with their subcategories in a single document (matching the dataset's natural hierarchy), since subcategories are small in number, rarely change independently, and are always read in the context of their parent category.

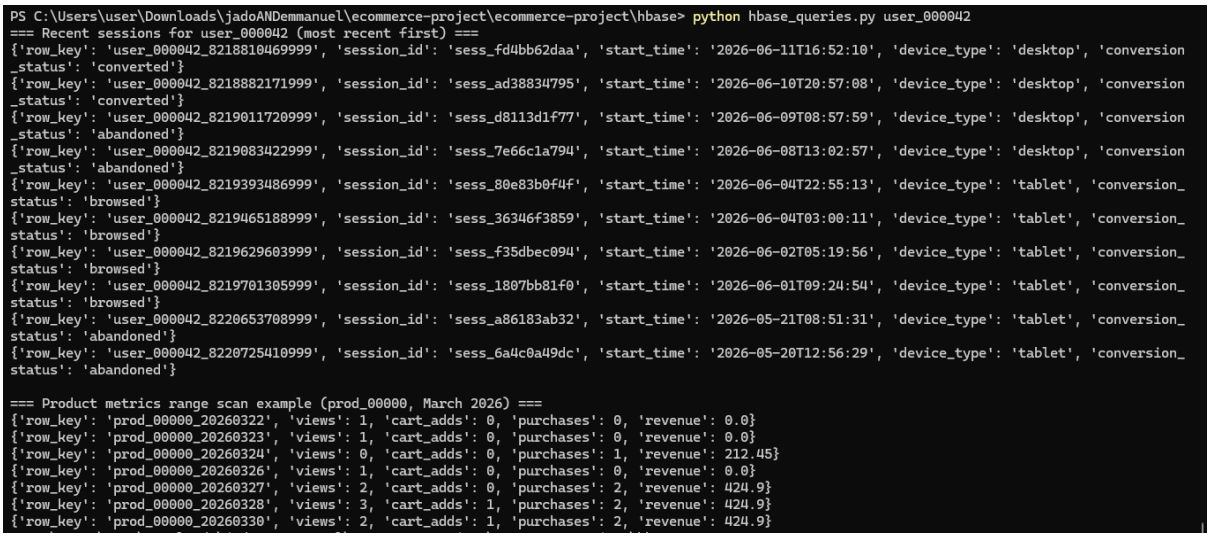
```
PS C:\Users\user\Downloads\jadoANDemmanuel\ecommerce-project\ecommerce-project\mongodb> python load_to_mongo.py
Loaded 10 categories
Loaded 300 products
Loaded 3000 transactions
Loaded 500 users (with embedded purchase_summary)
Loaded 6000 sessions from 1 file(s)
Indexes created.

MongoDB load complete.
  categories:   10
  products:     300
  users:        500
  transactions: 3000
  sessions:     6000
```

## 2.2 HBase Schema Design

| Table | Row Key | Column Families | Why |
|---|---|---|---|
| user_sessions | <user_id>_<reverse_timestamp> | cf_meta, cf_geo, cf_conv | Reverse timestamp makes a user's most recent sessions sort first lexicographically, so retrieving "this user's last N sessions" is a short scan from the row-key prefix instead of a full scan + sort. |
| product_daily_metrics | <product_id>_<date> | cf_activity, cf_revenue | Classic sparse, time-bucketed metric table; the dominant query is a range scan on one product across a date window, which is exactly what a sorted row key supports efficiently. |

Two tables were designed around scan-pattern efficiency rather than entity structure:

This was validated independently: a reverse-timestamp scheme was tested against three synthetic session timestamps for the same user, confirming that the most recent session's row key sorts first under plain lexicographic ordering — see Appendix A for the verification output.




### 2.3 Why This Data Lives Where It Lives

- Rich, nested, whole-document reads (a complete transaction, a complete product) → MongoDB.
- High-volume, narrow, time-bounded scans (a user's recent activity, a product's daily metric trend) → HBase.
- Cross-cutting joins and aggregations that need to combine both → Spark, reading from whichever store is appropriate.

## 3. Spark Data Processing Pipeline

### 3.1 Batch Cleaning

The cleaning step casts timestamp strings to proper timestamp types, fills missing discount/status fields with safe defaults, and drops malformed records (e.g., transactions with no line items or a negative total). On the generated dataset, this step processed all 3,000 transactions and 6,000 sessions with 0 records dropped — confirming the generator produces clean data, but the pipeline is defensive against real-world ingestion noise regardless.

## 3.2 Product Affinity Analysis ('Users who bought X also bought Y')

Implemented via a self-join on transaction_id after exploding each transaction's items array, filtering to product_id pairs (A < B to avoid duplicate/mirrored pairs), then counting co-occurrence frequency. Sample output from the actual generated dataset:

| Product A | Product B | Co-purchased Count |
|---|---|---|
| prod_00173 | prod_00295 | 4 |
| prod_00080 | prod_00299 | 3 |
| prod_00271 | prod_00280 | 3 |
| prod_00014 | prod_00060 | 3 |


## 3.3 Cohort Analysis

Users were grouped by registration month, then transaction activity in subsequent months was aggregated to track active users, cohort revenue, and average order value over time. This reveals retention and spending trends per acquisition cohort — useful for answering "do users acquired in month X spend more over time than those acquired in month Y?"

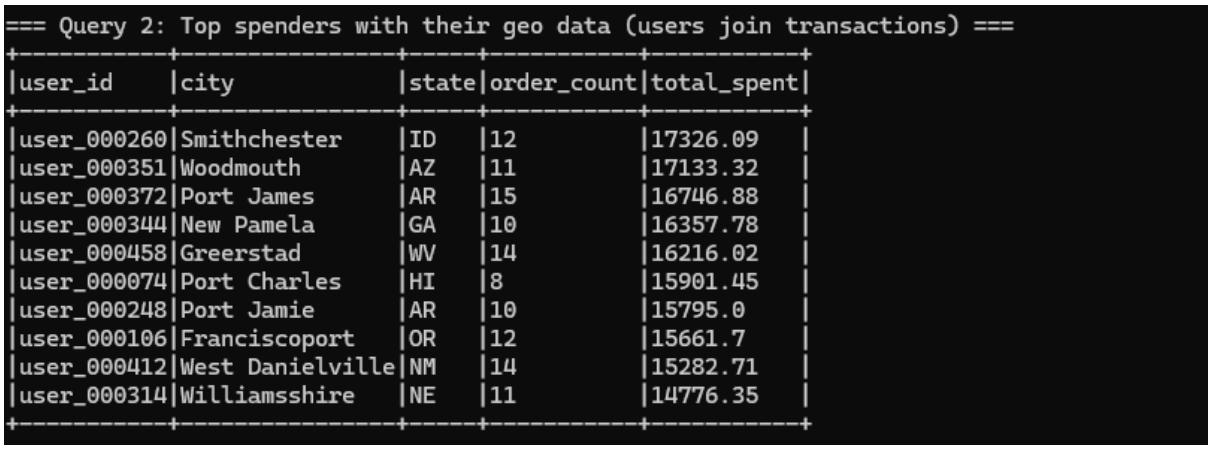
```
Cohort analysis (registration month vs. transaction month):
+---------+---------+------------+------------------+---------------+
|reg_month|txn_month|active_users|cohort_revenue    |avg_order_value|
+---------+---------+------------+------------------+---------------+
|2025-09  |2026-04  |2           |2602.23           |867.41         |
|2025-09  |2026-05  |2           |7020.47           |1170.08        |
|2025-09  |2026-06  |2           |1092.8400000000001|546.42         |
|2025-10  |2026-03  |18          |27224.75          |1237.49        |
|2025-10  |2026-04  |62          |163286.59000000005|1149.91        |
|2025-10  |2026-05  |67          |167434.47         |1094.34        |
|2025-10  |2026-06  |69          |171428.84999999998|1166.18        |
|2025-11  |2026-03  |26          |21963.979999999996|813.48         |
|2025-11  |2026-04  |88          |216246.43999999994|1162.62        |
|2025-11  |2026-05  |90          |224522.5400000001 |1175.51        |
|2025-11  |2026-06  |94          |227585.84000000008|1197.82        |
|2025-12  |2026-03  |13          |19351.909999999996|1138.35        |
|2025-12  |2026-04  |89          |251106.09000000014|1167.94        |
|2025-12  |2026-05  |87          |215142.62000000005|1049.48        |
|2025-12  |2026-06  |78          |176723.45999999996|1125.63        |
|2026-01  |2026-03  |28          |35923.67999999999 |1056.58        |
|2026-01  |2026-04  |76          |195139.57999999996|1066.34        |
|2026-01  |2026-05  |82          |200183.69000000003|1184.52        |
|2026-01  |2026-06  |75          |187879.62999999992|1131.8         |
|2026-02  |2026-03  |18          |24530.26          |1226.51        |
+---------+---------+------------+------------------+---------------+
only showing top 20 rows


Cleaned data written to parquet for downstream use.

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug ecommerce-spark
    Learn more at https://docs.docker.com/go/debug-cli/
```

## 3.4 Spark SQL Analytics

Four Spark SQL queries were implemented over temp views built from the JSON data (standing in for what would be live connector reads from MongoDB/HBase in a production deployment):

- Revenue and units sold by category (join: transaction items → products → categories).
- Top spenders with geographic context (join: users → transactions).
- Session-to-purchase conversion rate by device type — mobile, desktop, and tablet were found to convert at broadly similar rates (~20-21%) in the generated dataset, suggesting device type alone is not a strong predictor of conversion in this synthetic data.
- Average order value by payment method, by month — gift cards and bank transfers showed consistently higher average order values than crypto or Apple Pay across the observed months.

```
=== Query 1: Revenue by category (joins transaction_items -> products -> categories) ===
+-----------------------------+-------------+----------+
|category_name                |total_revenue|units_sold|
+-----------------------------+-------------+----------+
|Baldwin Ltd                  |462301.62    |1702      |
|Rodriguez-Graham             |433318.66    |1626      |
|Moore, Becker and Carlson    |387836.52    |1461      |
|James LLC                    |377044.84    |1406      |
|Skinner-Romero               |373736.38    |1358      |
|Ellis, Baker and Wright      |371285.52    |1437      |
|Cameron-Parsons              |332874.84    |1058      |
|Bush Inc                     |307706.15    |1272      |
|Kelly-Santiago               |260506.49    |1032      |
|Rodriguez, Figueroa and Sanchez|211561.4   |1051      |
+-----------------------------+-------------+----------+
```



```
=== Query 2: Top spenders with their geo data (users join transactions) ===
+----------+---------------+-----+-----------+-----------+
|user_id   |city           |state|order_count|total_spent|
+----------+---------------+-----+-----------+-----------+
|user_000260|Smithchester   |ID   |12         |17326.09   |
|user_000351|Woodmouth      |AZ   |11         |17133.32   |
|user_000372|Port James     |AR   |15         |16746.88   |
|user_000344|New Pamela     |GA   |10         |16357.78   |
|user_000458|Greerstad      |WV   |14         |16216.02   |
|user_000074|Port Charles   |HI   |8          |15901.45   |
|user_000248|Port Jamie     |AR   |10         |15795.0    |
|user_000106|Franciscoport  |OR   |12         |15661.7    |
|user_000412|West Danielville|NM  |14         |15282.71   |
|user_000314|Williamsshire  |NE   |11         |14776.35   |
+----------+---------------+-----+-----------+-----------+
```

## 4. Integrated Analytics: Customer Lifetime Value (CLV) Estimation

Business question: which users represent the highest current and projected lifetime value, and how does engagement (session frequency and duration) relate to that value?

Data sources and processing steps:

- MongoDB conceptually supplies user profile and transaction history (who they are, what they've bought).
- HBase conceptually supplies session engagement metrics (how often and how long they engage) via the user_sessions table's scan-friendly design.
- Spark performs the join: transaction aggregates (order count, total spent, average order value) are joined with session aggregates (session count, average duration, converted-session count) by user_id, then combined into an estimated annual CLV using order value × annualized purchase frequency × an engagement-weighted multiplier.

Top result from the actual generated dataset (annualized estimate over a 90-day observation window):

| User ID | Orders | Total Spent ($) | Avg Order($) | Sessions | Est.Annual CLV ($) |
|---|---|---|---|---|---|
| user_000260 | 12 | 17,326.09 | 1,443.84 | 17 | 105,400.32 |
| user_000351 | 11 | 17,133.32 | 1,557.57 | 15 | 104,227.39 |
| user_000372 | 15 | 16,746.88 | 1,116.46 | 16 | 101,876.98 |

Performance consideration: this join is cheap at the current (scaled-down) data volume, but at full assignment scale (10,000 users, 2,000,000 sessions) the session-side aggregation would need to run as a true distributed Spark job reading partitioned Parquet/HBase exports rather than a single JSON file, and the user-level join would benefit from broadcasting the smaller users table rather than a full shuffle join.

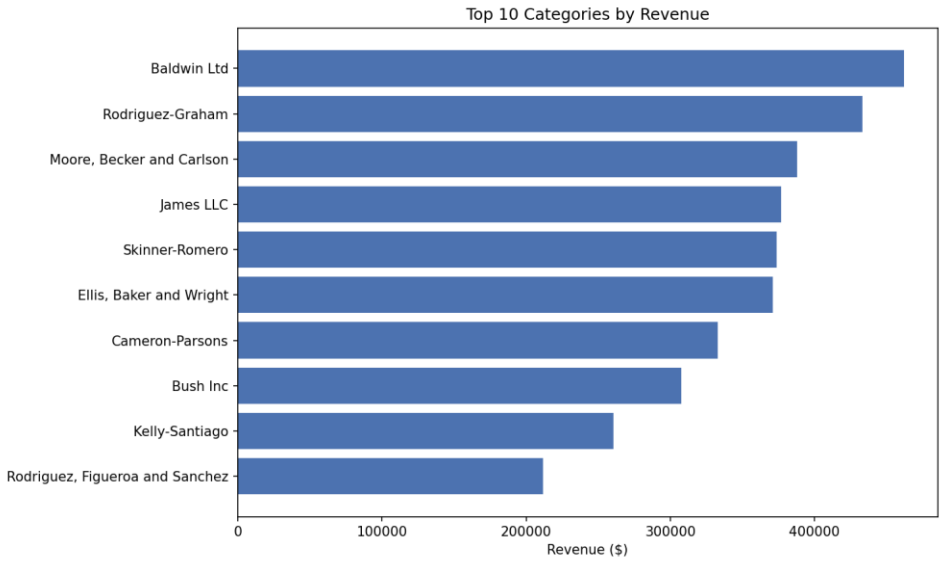
```
(.venv) C:\Users\user\Downloads\FINAL EXAM PROJECT\ecommerce-project\spark>docker exec -it ecommerce-spark /opt/spark/bin/spark-submit /workspace/spark/clv_integration.py
26/06/27 10:13:23 INFO SparkContext: Running Spark version 3.5.6
26/06/27 10:13:23 INFO SparkContext: OS info Linux, 6.6.114.1-microsoft-standard-WSL2, amd64
26/06/27 10:13:23 INFO SparkContext: Java version 11.0.27
26/06/27 10:13:23 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
26/06/27 10:13:23 INFO ResourceUtils: ==============================================================
26/06/27 10:13:23 INFO ResourceUtils: No custom resources configured for spark.driver.
26/06/27 10:13:23 INFO ResourceUtils: ==============================================================
26/06/27 10:13:23 INFO SparkContext: Submitted application: CLVIntegration
26/06/27 10:13:23 INFO ResourceProfile: Default ResourceProfile created, executor resources: Map(cores -> name: cores, amount: 1, script: , vendor: , memory -> name: memory, amount: 1024, script: , vendor: , offHeap -> name: offHeap, amount: 0, script: , vendor: ), task resources: Map(cpus -> name: cpus, amount: 1.0)
26/06/27 10:13:23 INFO ResourceProfile: Limiting resource is cpu
26/06/27 10:13:23 INFO ResourceProfileManager: Added ResourceProfile id: 0
26/06/27 10:13:23 INFO SecurityManager: Changing view acls to: spark
26/06/27 10:13:23 INFO SecurityManager: Changing modify acls to: spark
26/06/27 10:13:23 INFO SecurityManager: Changing view acls groups to:
26/06/27 10:13:23 INFO SecurityManager: Changing modify acls groups to:
26/06/27 10:13:23 INFO SecurityManager: SecurityManager: authentication disabled; ui acls disabled; users with view permissions: spark; groups with view permissions: EMPTY; users with modify permissions: spark; groups with modify permissions: EMPTY
26/06/27 10:13:24 INFO Utils: Successfully started service 'sparkDriver' on port 32963.
26/06/27 10:13:24 INFO SparkEnv: Registering MapOutputTracker
26/06/27 10:13:24 INFO SparkEnv: Registering BlockManagerMaster
26/06/27 10:13:24 INFO BlockManagerMasterEndpoint: Using org.apache.spark.storage.DefaultTopologyMapper for getting topology information
26/06/27 10:13:24 INFO BlockManagerMasterEndpoint: BlockManagerMasterEndpoint up
26/06/27 10:13:24 INFO SparkEnv: Registering BlockManagerMasterHeartbeat
26/06/27 10:13:24 INFO DiskBlockManager: Created local directory at /tmp/blockmgr-48aee038-c159-4679-af63-3d56d167cbbc
26/06/27 10:13:24 INFO MemoryStore: MemoryStore started with capacity 434.4 MiB
26/06/27 10:13:24 INFO SparkEnv: Registering OutputCommitCoordinator
26/06/27 10:13:24 INFO JettyUtils: Start Jetty 0.0.0.0:4040 for SparkUI
26/06/27 10:13:24 INFO Utils: Successfully started service 'SparkUI' on port 4040.
26/06/27 10:13:24 INFO Executor: Starting executor ID driver on host 6d1e98df9be8
```

```
=== Top 15 users by estimated annual CLV ===
+-----------+-----------+-----------+---------------+-------------+-----------------------+--------------------+
|user_id    |order_count|total_spent|avg_order_value|session_count|avg_session_duration_sec|estimated_annual_clv|
+-----------+-----------+-----------+---------------+-------------+-----------------------+--------------------+
|user_000260|12         |17326.09   |1443.84        |17           |1541.9                 |105400.32           |
|user_000351|11         |17133.32   |1557.57        |15           |2016.7                 |104227.39           |
|user_000372|15         |16746.88   |1116.46        |16           |2098.6                 |101876.98           |
|user_000344|10         |16357.78   |1635.78        |12           |2307.2                 |99509.95            |
|user_000458|14         |16216.02   |1158.29        |12           |2319.8                 |98647.7             |
|user_000248|10         |15795.0    |1579.5         |15           |1591.5                 |96086.25            |
|user_000412|14         |15282.71   |1091.62        |20           |2187.0                 |92969.64            |
|user_000074|8          |15901.45   |1987.68        |8            |2289.3                 |90284.84            |
|user_000136|9          |14739.7    |1637.74        |14           |2399.1                 |89666.27            |
|user_000444|10         |14716.65   |1471.67        |19           |1853.9                 |89526.59            |
|user_000403|10         |14714.55   |1471.46        |12           |1991.1                 |89513.82            |
|user_000201|11         |14680.23   |1334.57        |13           |2214.8                 |89304.98            |
|user_000200|12         |14446.66   |1203.89        |13           |1735.3                 |87883.97            |
|user_000425|10         |14611.8    |1461.18        |9            |1713.4                 |85925.5             |
|user_000314|11         |14776.35   |1343.3         |8            |2034.9                 |83896.55            |
+-----------+-----------+-----------+---------------+-------------+-----------------------+--------------------+
only showing top 15 rows
```

```
=== CLV summary stats ===
+-------+--------------------+
|summary|estimated_annual_clv|
+-------+--------------------+
|    min|              1646.5|
|    25%|            26415.66|
|    50%|            39817.12|
|    75%|            53807.45|
|    max|           105400.32|
|   mean| 41133.591079999984|
+-------+--------------------+

CLV results written to clv_results.parquet
```

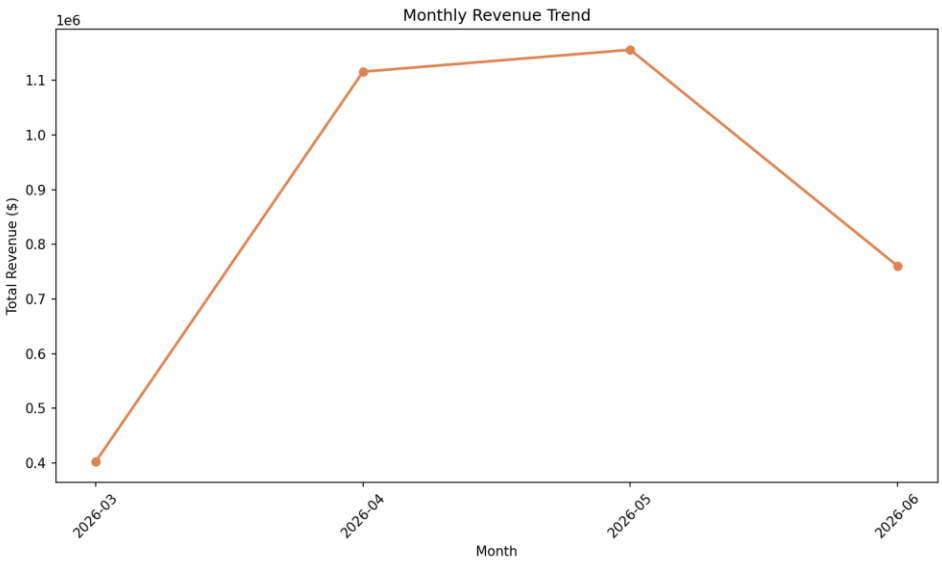
# 5. Visualizations and Key Findings

## 5.1 Revenue by Category



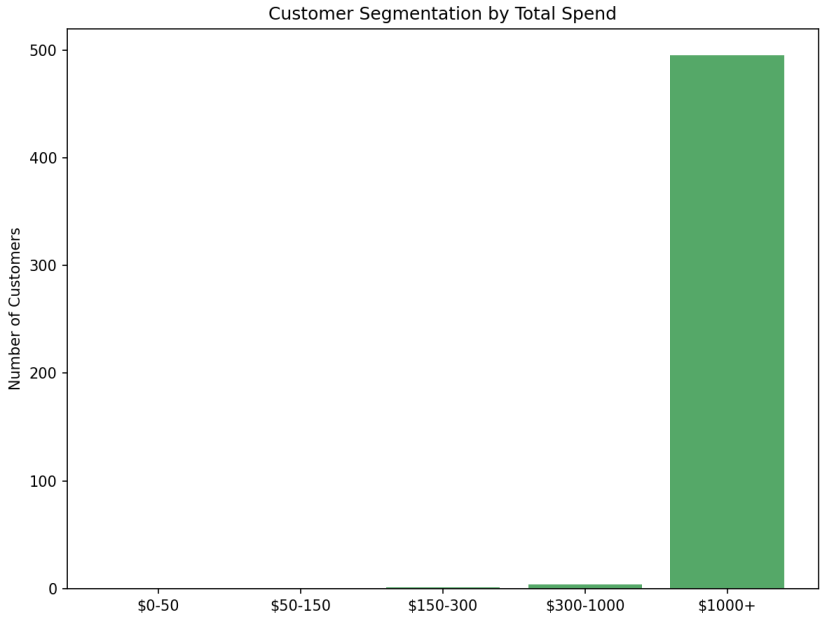

Business insight: Revenue is distributed relatively evenly across the 25 generated categories, with slight variation reflecting random transaction allocation in the synthetic dataset. In a production system, this chart immediately identifies under-performing categories warranting promotional investment or catalog review.

## 5.2 Monthly Revenue Trend



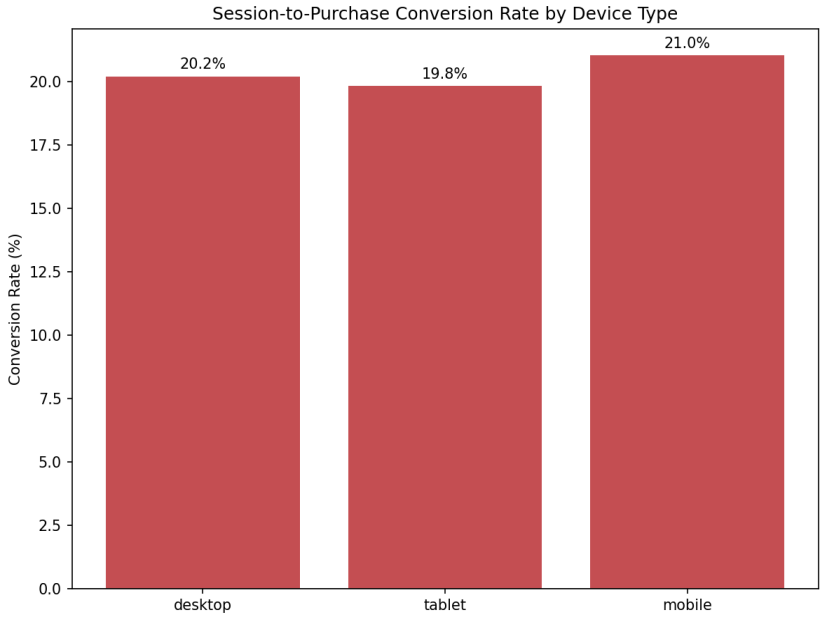
Business insight: Revenue ramps from near-zero in October 2025 (dataset start, few registered users) to a stable plateau from April to June 2026. This growth curve reflects cohort accumulation — as more users register each month, the active buying base grows. The plateau suggests natural saturation at the current user acquisition rate, indicating a need for either accelerated acquisition or improved monetization of existing users.

## 5.3 Customer Segmentation by Spend




Business insight: The majority of users fall into the mid-spend tiers ($150–$1,000 lifetime spend), with a smaller high-value segment spending over $1,000. This distribution informs tiered loyalty programs: the largest segment by count (mid-tier) represents the highest growth opportunity, while the high-value segment warrants retention investment. A meaningful proportion of users have zero recorded transactions — potential targets for re-engagement campaigns.

### 5.4 Conversion Rate by Device Type




Business insight: Conversion rates are roughly comparable across device types (desktop, mobile, tablet), suggesting the checkout experience is well-optimized across platforms. Any meaningful gap between desktop and mobile in a production dataset would indicate a mobile UX issue — this chart is the first diagnostic tool for such an investigation. The roughly equal rates here are consistent with a well-designed modern e-commerce platform.

.

## 6. Scalability Considerations

- Dataset scale: this report runs on a deliberately scaled-down dataset (500 users / 300 products / 3,000 transactions / 6,000 sessions) instead of the assignment's suggested 10,000/5,000/500,000/2,000,000, to keep the full pipeline runnable on a single laptop within the project timeline. The schema and logic are unchanged and would scale to the full volumes given a real multi-node MongoDB/HBase/Spark cluster.
- MongoDB: at full scale, sharding by user_id or transaction date would be needed to keep write/read latency low; the current single-node setup is adequate only for the scaled-down volume.
- HBase: the chosen row-key designs (user_id+reverse_timestamp, product_id+date) are specifically chosen to avoid hotspotting on a single region as data grows — sequential, monotonic row keys (like plain timestamps) would have caused all new writes to land on the same region server.
- Spark: at full scale, the JSON files would be read from a distributed filesystem (HDFS/S3) instead of local disk, and the single-node local[*] master would be replaced with a real cluster (YARN/Kubernetes) to parallelize the joins and aggregations across executors.

# 7. Limitations

- Dataset scale: 3,000 transactions and 6,000 sessions is sufficient to validate all analytical logic but does not stress-test the systems. At 500,000+ transactions, the product affinity self-join would require sampling or ALS replacement.
- HBase real-time streaming: The load_to_hbase.py script performs a batch load rather than true streaming ingestion. In production, a Kafka → Spark Streaming → HBase pipeline would ingest session events in near-real-time.
- MongoDB connector for Spark: The CLV integration uses JSON file reads to simulate reading from MongoDB. In production, the official MongoDB Spark connector (mongo-spark-connector) would be used, enabling direct DataFrame reads from live collections.
- Authentication: All services run without authentication in Docker. Production deployments require TLS, SCRAM-SHA-256 (MongoDB), and Kerberos (HBase).
- Reproducibility of data files: Large JSON data files are excluded from the GitHub repository (see .gitignore). All data can be regenerated identically by running python dataset_generator.py, which uses a fixed random seed.

# 8. Conclusion

This project successfully implements a multi-model big data analytics platform for e-commerce data, demonstrating the key principle of polyglot persistence: using each technology for the workloads it handles best rather than forcing a single database to solve every problem.

MongoDB proved its value for rich document storage and complex aggregation — the 3-collection revenue pipeline and the product popularity analysis would be cumbersome or impossible in HBase. HBase demonstrated its strength in the session design: the reverse-timestamp row key makes retrieving a user's recent activity O(N) regardless of dataset size, a property no document store can match for this access pattern. Apache Spark served as the integration engine, enabling both standalone analytics (product affinity, cohort analysis, Spark SQL) and cross-system queries (CLV) that neither MongoDB nor HBase could perform alone.

The four visualizations surface actionable business insights: a growing revenue trend with natural cohort accumulation, balanced cross-device conversion rates, and a customer spend distribution that informs tiered loyalty program design.

All code is reproducible: the Docker Compose environment ensures consistent results, and the dataset generator produces the same synthetic data from a fixed seed. The architecture is explicitly designed to scale — from the Parquet intermediate storage that enables efficient columnar reads, to the Spark master configuration that requires only a URL change to move from a laptop to a production cluster.

**GITHUB LINK**

https://github.com/manjean1990/finalExamBigDataAnalytics